ReviewIQ

Technical Viva Guide

Vidyalankar Institute of Technology | Guide: Prof. Sneha Annappanavar

The 30-second pitch (anyone can say this)

Product reviews are unstructured text, and a star average hides detail. ReviewIQ takes 50,000 Amazon Electronics reviews and builds three layers on the same data: **NLP** to understand the language (sentiment, topics, aspects), **deep learning** to classify sentiment with a BiLSTM and a fine-tuned DistilBERT, and **advanced ML** to score helpfulness, cluster reviews and flag suspicious ones. A semantic-search question answering layer answers only from a product's own reviews and refuses when evidence is weak. Everything is in one Streamlit dashboard.

Who presents what (swap names freely)

Speaker	Part	Covers
Nidhi Bhawari (23102A0006)	1. Data and NLP	Problem, dataset, labels, split, VADER, TF-IDF, LDA, aspect sentiment
Aarti Sakpal (23102A0007)	2. Deep learning	BiLSTM with attention, DistilBERT, results, ablation, cost
Gayatri Patil (23102A0016)	3. Advanced ML	XGBoost, KMeans, Isolation Forest
Prachi Wakode (23102A0020)	4. Search, app, demo	Embeddings, RAG guardrail, dashboard, demo, limits

Order of the live demo (Prachi drives it, each speaker points at their part)

1. **Home:** three pillars and pipeline (everyone).
2. **Explore Products:** search "echo", open *Echo Dot* with View AI Insights.
3. **Product Insights:** Aspect Analysis tab, click an aspect to show evidence sentences (Nidhi). Review Trust tab, drag the slider (Gayatri).
4. **Ask ReviewIQ:** "Is this good for cutting cable TV?" on the first Fire TV Stick, then "Does it have a 3 year warranty?" to show the refusal (Prachi).
5. **Deep Learning Lab, Try it live:** "Mixed (aspect-level)" example, click Classify review. The three models disagree and the attention words light up (Aarti).
6. **Business Intelligence, Complaint tab:** battery is the top complaint (Nidhi).

Five numbers every speaker must know

50,000 reviews, 34,365 products

88.8% / **0.718** BiLSTM accuracy / macro-F1

89.9% DistilBERT accuracy (6,000 train)

57× LSTM faster on CPU, 26× fewer params

20.2% battery mentions negative (top complaint)

Golden rules for the viva

- Say **"agreement with star ratings"**, not "true accuracy": the labels are weak (from stars).
- Say **"potentially suspicious"**, never "fake". That module is unsupervised, so it has no accuracy figure.
- If asked "is the LSTM better than DistilBERT?": better macro-F1 (0.718 vs 0.702) but **not statistically significant**; DistilBERT is better on accuracy (significant). DistilBERT saw only 6,000 reviews.
- If you do not know a detail, say what the code does, not what you guess. Everything here is in `src/`.

Nidhi Bhawari 23102A0006

Part 1: Problem, data and NLP (about 2.5 min)

Problem. Reviews are free text, so a 4.4 star average cannot tell a buyer that the sound is great and the battery is poor. We ask three questions: should I buy this, what should be improved, can I trust these reviews.

Data. Amazon Reviews 2023, Electronics: a 50,000 review sample, 34,365 products, 8,883 reviewers. Labels come from stars: 4 and 5 positive, 3 neutral, 1 and 2 negative, giving 80.4%, 7.0%, 12.6%. Because it is so imbalanced we use **macro-F1**. I cleaned it (HTML and URLs stripped, fewer than 3 words dropped, duplicate user-product-text removed) and joined product name, brand and image by `parent_asin` from a 5.2 GB metadata file (34,364 of 34,365 matched). Every model uses one **stratified 80/20 split, seed 42**: 40,000 train, 10,000 test.

NLP. Four components. **VADER** is a lexicon scorer; compound ≥ 0.05 positive, ≤ -0.05 negative. **TF-IDF** with unigrams and bigrams, 30,000 features, sublinear tf $(1 + \log \text{tf})$, feeds a logistic regression and a linear SVM, both class-balanced. **LDA** finds 8 topics. **Aspect sentiment** finds what customers feel about each feature.

Technical detail to keep ready

- **TF-IDF + SVM:** TF-IDF with 30,000 features, n-grams (1,2), `min_df`=2, `sublinear_tf`=True; `LinearSVC` with `class_weight`=balanced; LR with `max_iter`=1000, balanced.
- **LDA:** count vectors with 3,000 features, bigrams, `min_df`=5, `max_df`=0.85, English stop words; online variational Bayes, 8 topics, 6 iterations, batch 1024; fitted on an 8,000-review subsample then applied to all 50,000 for speed.
- **Aspect sentiment (ABSA):** sentence split with a regex on sentence-ending punctuation; 10 keyword lexicons (battery, camera, display, sound, performance, build quality, price, connectivity, size, customer service); the polarity of an aspect is VADER on the concatenation of the sentences that mention it; the matched sentence is stored as **evidence**. Result: 64,134 mentions in 33,257 reviews; **battery 20.2% negative**, connectivity and customer service 19.3%, build quality 8.1%.
- **Test results (10,000 reviews):** VADER .813 / .512, LR .860 / .689, SVM .888 / .693 (accuracy / macro-F1). VADER neutral F1 is only 0.08.

Likely questions

Why macro-F1 and not accuracy? Predicting "positive" for everything already gives about 80% accuracy. Macro-F1 averages the three class F1 scores equally, so the 7% neutral class counts as much as the 80% positive class.

What is TF-IDF? A word's weight is its frequency in the review times how rare it is across reviews, so common words matter less. Sublinear tf dampens repeated words.

Why is the sample biased? We took a contiguous block of the source file, which is grouped by reviewer (9,686 reviewers in 65,552 raw records), so prolific reviewers are over-represented. It matters for the suspicious-review feature.

Is the aspect module validated? Not against human labels. It is keyword matching plus VADER, so it misses implicit aspects and sarcasm. We say this in the limitations.

Why does stratifying matter? It keeps the 80/7/13 class ratio identical in train and test, so a rare class cannot vanish from the test set.

Aarti Sakpal 23102A0007

Part 2: Deep learning (about 2.5 min)

BiLSTM with attention. Text is lower-cased and tokenised with the regex `[a-z0-9']+ | [!?]`. The vocabulary is built on the training split only (14,937 tokens, minimum frequency 3) and reviews are truncated to 200 tokens. A learned 128-d embedding feeds a **2-layer bidirectional LSTM**, 128 units per direction, so each token gets a 256-d vector that sees both left and right context. **Attention pooling** scores every token, $\alpha_t = \text{softmax}(\mathbf{w}^\top \mathbf{h}_t)$ with padding masked, and the sentence vector is $\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t$. Then dropout 0.4 and a linear layer to 3 classes: **2,572,420 parameters**, about 7 minutes on an RTX 4050.

Imbalance. The loss is cross-entropy with class weights $w_c = \sqrt{N/(3n_c)}$, and we stop on **validation macro-F1**, patience 3 (best epoch 6 of 9).

DistilBERT. Pre-trained 6-layer transformer, 66.96 M parameters, fine-tuned with a 3-way head: AdamW 2×10^{-5}, 2 epochs, batch 8, max length 128, on a 6,000-review subsample on CPU (about 60 minutes).

Technical detail to keep ready

- **Training:** Adam, lr 2×10^{-3}, weight decay 10^{-5}, batch 128, gradient clipping 1.0, packed sequences (`pack_padded_sequence`) so padding is ignored by the LSTM. Split 36,000 train / 4,000 validation / 10,000 test.
- **Results:** BiLSTM .888 / **.718**; DistilBERT **.899** / .702. Neutral F1: LSTM .446, SVM .382, DistilBERT .368.
- **Significance (2,000 bootstrap resamples):** LSTM minus SVM macro-F1 $+0.025$, CI $[+0.010, +0.039]$, significant. LSTM minus DistilBERT $+0.016$, CI $[-0.000, +0.031]$, not significant. DistilBERT accuracy lead $+0.011$ is significant.
- **Ablation (3 seeds):** full .891 / .722; no class weights .899 / .709; mean pooling .889 / .715. So class weights give $+0.012$ macro-F1 for -0.007 accuracy; attention adds only about $+0.007$. Seed spread is about 0.004.
- **Cost:** 26$\times$ fewer parameters, 57$\times$ faster on CPU (975 vs 17 reviews/s), 13$\times$ on GPU.
- **Word clouds:** word weight $= f_{class} \cdot (f_{class}/f_{all})^{0.7}$, so distinctive words win over merely frequent ones.

Likely questions

Why an LSTM at all? It models word order and negation (“not good”) that bag-of-words cannot, is trained from scratch in minutes, and is cheap to serve. It is the course’s required DL model.

What does attention do? It learns a weight per word so the sentence vector focuses on informative words. You can see it in the live demo: “terrible” and “amazing” are highlighted.

Why $\sqrt{\cdot}$ in the class weight? Full inverse-frequency weights over-correct and hurt accuracy; the square root is a milder compromise. We did not tune it, which we state.

Is DistilBERT unfairly treated? Partly: 6,000 reviews versus 36,000 for the LSTM, and no class weights. That is a stated limitation and the first item in future work.

Gayatri Patil 23102A0016

Part 3: Advanced machine learning (about 2.5 min)

XGBoost helpfulness. The target is whether a review got at least one helpful vote (26.2% of reviews). Ten engineered features: rating, review length, title length, verified purchase, exclamation and question counts, capital-letter ratio, VADER compound, review age, and **number of aspects mentioned**, which comes from the NLP module. I tuned it with `RandomizedSearchCV`, 15 candidates, 3-fold, scored on F1, with `scale_pos_weight` for the imbalance. Best: 200 trees, depth 6, learning rate 0.01, subsample 0.7. Result: **70.5% accuracy**, helpful-class precision .458, recall .702, F1 .555.

KMeans. MiniBatchKMeans, $k = 6$, on a separate TF-IDF space. Honest finding: only 2 clusters are topical, the other 4 are generic praise.

Isolation Forest. Unsupervised, because we have no fake-review labels. It builds random trees; anomalies are isolated in few splits, giving score $s(x) = 2^{-E[h(x)]/c(n)}$. Six features go through a StandardScaler; contamination 0.10 flags **4,999 reviews**.

Technical detail to keep ready

- **XGBoost confusion matrix** (rows true, cols predicted): unhelpful 5211 / 2172, helpful 779 / 1838. Top features: review length, aspects mentioned, rating. Accuracy is modest because helpfulness depends on how many people saw the review, which no review feature captures.
- **KMeans:** `MiniBatchKMeans(n_clusters=6, n_init=10, batch_size=1024)` on a 5,000-feature TF-IDF with stop words removed. Sizes 19,309 / 3,570 / 13,552 / 3,228 / 5,101 / 5,240. Topical: laptop bags and cases, sound and headphones.
- **Isolation Forest features:** is_short (< 10 words), capital ratio, exclamation count, verified flag, $\log(1+\text{reviews})$, and max cosine similarity to another review of the *same product*, computed per product group on the MiniLM embeddings (a full $50{,}000^2$ matrix would be 2.5 billion pairs). Score is $-$`score_samples` min-max scaled to $[0, 1]$; reasons are rule-based strings. 36 reviews have similarity ≥ 0.95.
- **The bug we fixed:** the first version used raw rating and raw length; scores correlated $+0.55$ with length and -0.42 with rating, so long detailed negative reviews were flagged. After switching to directional features the correlations became $+0.06$ (length) and $+0.25$ (account volume).
- **UI:** a threshold slider re-flags reviews live; the table shows flag, score, rating, review text and signals.

Likely questions

How do you know the flags are right? We do not, and we say so. With no ground truth there is no accuracy; we inspected top flags (templated one-liners, ALL CAPS, high-volume unverified accounts) and call them “potentially suspicious”.

Why contamination 0.10? It is an assumption about the anomaly rate, not a measurement; the slider lets users pick their own cut-off.

Caveat on account volume? The source file is reviewer-grouped, so counts per reviewer in our sample are inflated.

What is `scale_pos_weight`? Ratio of negative to positive training examples; it makes XGBoost penalise missing a helpful review more.

Prachi Wakode 23102A0020

Part 4: Search, question answering, dashboard and demo (about 2.5 min)

Semantic search. Every review is encoded once with **all-MiniLM-L6-v2** into a 384-d unit vector and stored, so cosine similarity is just a dot product. For a question about a product we mask to that product's reviews (`parent_asin`) and take the **top 5**. **FLAN-T5-base** is prompted with the question and those reviews, truncated to 300 characters each, and answers in 1 to 2 sentences, greedy decoding.

The guardrail. Small generators do not obey "say you do not know", so we added a hard rule: if the best similarity is below **0.30** the generator is never called and the answer is "not enough information". Measured over 9 products and 45 questions per group: **97.8%** of answerable questions accepted, **95.6%** of plausible but unanswerable and **100%** of off-topic questions declined. Dense retrieval also beat TF-IDF on paraphrased aspect queries, 0.54 vs 0.32.

Dashboard. Streamlit, nine pages. Data logic is in pure functions; heavy work is cached. Summaries and comparisons are **templates built from computed numbers, never an LLM**, so they cannot invent facts.

Technical detail to keep ready

- **Why scope by product?** Otherwise reviews of product B can answer a question about product A. Retrieval is restricted to the selected product's reviews, so evidence from other products cannot appear.
- **Caching:** `@st.cache_data` for tables and features, `@st.cache_resource` for models. Without it, the helpfulness view recomputed VADER on 50,000 reviews (about 23 s) on every click.
- **Testing:** Streamlit `AppTest` runs all 9 pages for exceptions; Playwright clicked through 12 sidebar transitions and both in-page buttons.
- **Bugs found and fixed:** backwards trust features, ungrounded answers, slow reruns, missing product images (wrong field name, dead fallback link), ignored first sidebar click (widget identity changed every page; fixed with a stable key and callbacks), and a folder named `pages` that Streamlit read as a second menu.
- **Retrieval test caveat:** 10 queries, relevance judged by a keyword lexicon (so a lower bound); the battery query scored 0 for both methods.
- **Limitations:** weak labels from stars; one English category in a reviewer-grouped sample; DistilBERT under-trained; keyword aspects; flags are anomalies; answer faithfulness was not measured.
- **Future work:** DistilBERT on full data with matched class weights, learned aspect extraction, human-annotated subsets, random multi-category sampling, a faithfulness check for answers.

Likely questions

What is RAG? Retrieve evidence first, then generate from it, so every answer can be traced to reviews. The user always sees the five retrieved reviews and their similarity scores.

Why does the model refuse sometimes? By design: when similarity is under 0.30 the evidence is too weak, which we measured separates answerable from unanswerable questions (mean 0.455 vs 0.218).

Closing line: "Three families of AI, one corpus, every insight linked to evidence, and the limits stated honestly."

Technical cheat sheet

keep this open during the viva

Term	One-line meaning
Macro-F1	Mean of the per-class F1 scores, $F1_c = 2P_cR_c/(P_c + R_c)$; weights rare and common classes equally
Stratified split	Same class ratio in train and test (80/7/13)
TF-IDF	Term frequency times inverse document frequency; down-weights common words
LDA	Probabilistic topic model: each review is a mixture of topics, each topic a distribution over words
VADER	Rule and lexicon sentiment scorer, compound score in $[-1, 1]$
LSTM / BiLSTM	Recurrent network with gates that keep long-range context; bidirectional reads both ways
Attention pooling	Learned weights over tokens: $\alpha_t = \text{softmax}(\mathbf{w}^\top \mathbf{h}_t)$, $\mathbf{c} = \sum \alpha_t \mathbf{h}_t$
Class weighting	Loss multiplied by $w_c = \sqrt{N/(3n_c)}$ so rare classes matter more
Early stopping	Stop when validation macro-F1 has not improved for 3 epochs, keep the best epoch
DistilBERT	Smaller transformer distilled from BERT; fine-tuned with a classification head
Bootstrap CI	Resample the test set 2,000 times; the 2.5th and 97.5th percentiles give the 95% interval
XGBoost	Gradient-boosted decision trees; each tree fits the errors of the previous ones
Isolation Forest	Random trees isolate outliers in fewer splits; shorter path means more anomalous
Cosine similarity	$\cos(q, r) = \mathbf{q} \cdot \mathbf{r}$ for unit vectors; 1 is identical meaning
RAG	Retrieval-augmented generation: retrieve evidence, then generate from it
Similarity floor	If best cos < 0.30, refuse instead of generating

All results on the same 10,000 test reviews

Model	Accuracy	Macro-F1	F1 neutral	Speed (CPU, reviews/s)
VADER	.813	.512	.080	n/a
TF-IDF + LR	.860	.689	.400	n/a
TF-IDF + SVM	.888	.693	.382	13,550
BiLSTM + Attention	.888	**.718**	**.446**	975
DistilBERT	**.899**	.702	.368	17

Files to know (all under `src/`)

`data_prep.py, metadata_prep.py`	Cleaning, labels, product metadata join
`sentiment/, topic_modeling/, absa/`	VADER, TF-IDF models, LDA, aspect sentiment
`deep_learning/lstm_sentiment.py, lstm_model.py`	BiLSTM model and training
`deep_learning/bert_finetune.py`	DistilBERT fine-tuning
`advanced_ml/`	XGBoost helpfulness, KMeans, Isolation Forest
`semantic_search/, rag_chatbot/`	Embeddings, retrieval, guardrail, answers
`dashboard/ (app.py, sections/)`	The 9-page Streamlit app
`evaluation/`	Bootstrap, ablation, cost, retrieval and guardrail experiments